

Part 1: Microservice Infrastructure (Lessons 6-8).

Service Registry & Discovery

- **What it is:** A "phone book" where services register their locations and metadata.
- **Problem solved:** It removes the need for services to know physical locations of others, enabling loose coupling and easy scaling.
- **Failover:** To avoid a single point of failure, registries are replicated in clusters (e.g., Consul or Eureka).

Load Balancing & API Gateway

- **Load Balancing:** Distributes requests across multiple service instances. Tools like Feign use the registry to perform "round-robin" balancing.
- **API Gateway:** Acts as a single entry point for external clients to prevent tight coupling with internal services. It handles cross-cutting concerns like routing, security, and logging.
- **High Availability:** Using active-passive or active-active configurations with a reverse proxy (like Nginx) avoids gateway failure.

Monitoring & Resilience

- **Tracing (Zipkin):** Monitors service-to-service communication. It uses **Trace IDs** for the entire request and **Span IDs** for individual operations.
- **Logging (ELK):** Centralizes logs because containers have no fixed address. **Logstash** collects, **Elasticsearch** searches, and **Kibana** visualizes data.
- **Circuit Breaker:** Prevents one slow service from slowing down the entire system. It transitions through states: **Closed** (normal), **Open** (fails fast to fallback), and **Half-Open** (testing if recovered).
- **Config Server:** Centralizes configuration to keep all services in sync.
- **Spring Cloud Contract:** Ensures consumer stubs stay in sync with the producer's API to prevent local tests from passing while production calls fail

Part 2: Data & Transaction Patterns (CQRS, Event Sourcing, and SAGA).

This part of the course focuses on how microservices manage data and maintain consistency without traditional, heavy-duty transactions.

CQRS (Command Query Responsibility Segregation)

- **What it is:** This pattern separates write activities (**Commands**) from read activities (**Queries**).
- **Problem solved:** It handles situations where database queries happen much more often than data changes or when read and write sides have different scaling needs.
- **How it works:** The command side uses a normalized model to ensure data consistency, while the query side uses a denormalized model (often in a fast NoSQL database) for high-performance reads.
- **Trade-off:** It provides great scalability and performance but increases costs and complexity because you have to manage multiple database technologies.

Event Sourcing

- **What it is:** Instead of storing the current state of an object, you store a continuous stream of immutable events that represent every change that ever happened.
- **Reconstructing State:** To find the current state, the system replays all events from the "beginning of time" or since the last midnight snapshot.
- **Advantages:** It provides a built-in audit log, allows you to "undo" or rewind changes, and avoids concurrent update issues.
- **Disadvantage:** Read performance is low because you must replay events; therefore, you should **always use CQRS** with Event Sourcing to provide a fast read view.

SAGA Pattern

- **What it is:** A pattern used to manage distributed transactions that involve multiple steps across different services.
- **Compensating Actions:** Since you can't "rollback" a distributed database easily, if one step fails, the Saga executes "undo" actions (compensating transactions) for all previously successful steps.
- **Implementation Styles:**
 - **Orchestration:** A central "brain" or process manager tells each service what to do and when.
 - **Choreography:** Services move the process forward by exchanging events (pub-sub) without a central coordinator.

Part 3: Security & Messaging (Lessons 10-11).

Security (Lesson 10)

- **OAuth2 & Authorization Server:** You use a central server (like Keycloak) to configure **realms**, **clients** (with IDs and secrets), and **users** (with roles).
- **JWT (JSON Web Token):** These tokens allow a receiver to verify correctness using a built-in **signature**.
- **API Gateway Integration:** The gateway can act as a gatekeeper, checking for valid tokens and forwarding them to secure internal web applications.

Kafka (Lesson 11)

- **Core Concepts:** Kafka is built for event sourcing. Key terms include the **Retention Period** (how long events are kept), **Offset** (the consumer's position in the stream), and **Consumer Groups** (allowing multiple consumers to share the workload).
- **Scaling & Reliability: Partitioning** allows you to scale by spreading data across brokers using **keys**, while **Replication** ensures no data is lost if a broker fails.
- **Error Handling:**
 - **Blocking Retry:** Stops processing the partition until the specific message succeeds or fails.
 - **Non-blocking Retry:** Moves failed messages to a separate retry topic so the main partition can keep moving.
 - **Dead Letter Topic:** A final destination for messages that cannot be processed after all retries.
- **Transactions:** Kafka supports transactions to ensure "exactly-once" semantics, meaning an event is either fully processed or not processed at all.

Part 4: Architectural Design and Diagrams.

Architectural characteristics (the "-ilities") are success criteria like performance, scalability, and security that define how a system should function, often independent of its specific business features. Designing the "right" architecture involves balancing these qualities against business constraints and functional requirements.

Risk Storming & Mitigation

Risk storming is a collaborative technique where team members identify risks on architectural diagrams using post-it notes. Each risk is quantified by its impact and likelihood (from 1 to 9). Mitigation involves changing the architecture—for example, adding multiple instances or Kafka partitioning to address elasticity risks—to reduce those threats.

The C4 Model & Diagrams

The C4 model is a hierarchical way to communicate architecture at different levels of abstraction:

- **Context Diagram:** Shows the "big picture" of how your system interacts with users and external systems.
- **Container Diagram:** Focuses on the high-level shape, showing separate deployable units like web servers, databases, and file systems and how they communicate.
- **Component Diagram:** Decomposes containers into distinct logical parts (e.g., "Security Component") to show their specific responsibilities.
- **Code (Class) Diagram:** The lowest level, showing how individual components are implemented at the code level.

For your exam, you should also be prepared to draw:

- **Use Case Diagrams:** To identify actors and functional goals.
- **Activity Diagrams:** To model the processing steps of a use case.
- **Sequence Diagrams:** To detail the communication between components over time